

Informe: Trabajo Práctico #2 - Arquitectura de Capas (Iteración 1)

Asignatura: Sistemas de Computación

Institución: Facultad de Ciencias Exactas, Físicas y Naturales (FCEFN) – UNC

Docente: Javier Alejandro Jorge

Datos del Grupo y Repositorio

- **Integrantes:**
 - Macarena Vanina González
 - Marcos Nieto
 - Mario Pampiglione
- **Repositorio:** https://github.com/Maca040/SistComp_TeamMergeNoConflict.git

1. Introducción y Arquitectura

Este trabajo práctico aborda el diseño de una aplicación distribuida en capas para procesar datos socioeconómicos reales. La arquitectura se compone de:

- **Capa Superior (Python):** Gestiona la obtención de datos desde la API REST.
- **Capa Intermedia (C):** Actúa como puente y gestiona la convención de llamadas.
- **Capa Inferior (Assembler):** Realiza el procesamiento a nivel de hardware.

2. Implementación Técnica: Iteración 1 (Python y C)

En la primera instancia, se implementó una solución que recupera el índice GINI de Argentina desde la API REST del Banco Mundial y lo procesa mediante una librería dinámica compilada en C.

2.1 Archivos de la Iteración 1

- **main.py:** Es el script principal que consume la API del Banco Mundial, busca el último índice GINI válido de Argentina y utiliza el módulo **ctypes** para invocar la librería en C y enviarle el valor.
- **calculadora.c:** Inicialmente (Iteración 1), este archivo recibía el valor flotante, realizaba el casteo a entero y sumaba +1.
- **libcalculadora.so:** Es el archivo objeto compartido (librería dinámica) generado tras compilar la capa C para que Python la ejecute.

Para demostrar la correcta comunicación y el pasaje de parámetros, adjuntamos la salida por consola original:

```
SistComp_TeamMergeNoConflict on  dev_Maca [?] via  v3.12.3 (.venv)
> /usr/bin/python3 /home/maca/Documentos/SIST_COMP/SistComp_TeamMergeNoConflict/TP2/sin_Assembler/main.py
Dato de la API -> GINI Argentina (2020): 42.7
[Capa C] Valor original recibido: 42.70
[Capa C] Valor convertido y sumado: 43
Resultado final devuelto por C (convertido + 1): 43
```

3. Implementación Técnica: Iteración 2 (Ensamblador y Stack)

Para la segunda iteración, se modificó la arquitectura delegando el procesamiento matemático a una rutina en lenguaje Ensamblador, dejando a C exclusivamente como puente.

3.1 Nuevos Archivos de la Iteración 2

- **calculadora_asm.asm**: Rutina en ensamblador que recibe el flotante en el registro `xmm0` (System V ABI), lo convierte a entero truncado (`cvtts2si`), le suma una unidad (`inc eax`) y devuelve el resultado. Implementa el correcto armado y desarmado del Stack Frame.
- **main_c.c y app_gdb**: Un programa en C puro y su ejecutable para aislar la capa inferior y validar el comportamiento de la memoria sin la intervención de Python.

3.2 Validación con GDB

Se utilizó el ejecutable de C puro (**app_gdb**) para validar el comportamiento de la pila y los registros en GDB. Se comprobó la correcta ejecución instrucción por instrucción (**stepi**), validando:

1. El armado del Stack Frame (**push rbp**).
2. El correcto pasaje del registro matemático (de **0x2a** a **0x2b** tras el **inc eax**).
3. La correcta limpieza de la pila al retornar.

```
(gdb) break procesar_gini_asm
Breakpoint 1 at 0x1210: file calculadora_asm.asm, line 7.
(gdb) run
Starting program: /home/marcos/repos/SistComp_TeamMergeNoConflict/TP2/sin_Assembler/scr/app_gdb
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
--- Iniciando prueba en C puro para GDB ---
[Capa C] Valor original recibido: 42.80

Breakpoint 1, procesar_gini_asm () at calculadora_asm.asm:7
7      push rbp                ; se guarda el Base Pointer de la función que nos llamó (C)
(gdb) print $xmm0.v4_float
$1 = {42.7999992, 0, 0, 0}
(gdb) stepi
8      mov rbp, rsp           ; el stack pointer actual se convierte en el nuevo base pointer para esta función
(gdb) x/8xw $sp
0x7fffffffdb80: 0xfffffbb0  0x00007fff  0x555551e4  0x00005555
0x7fffffffdb90: 0x00000001  0x00000000  0x00000000  0x422b3333
(gdb) stepi
12     cvtts2si eax, xmm0     ; se convierte el float en xmm0 a un entero en eax, truncando hacia cero.
(gdb) stepi
13     inc eax                ; se suma 1 al resultado entero.
(gdb) info registers rax
rax    0x2a                42
(gdb) stepi
16     mov rsp, rbp          ; se restaura el Stack Pointer al valor del Base Pointer, limpiando el stack frame de esta función
(gdb) info registers rax
rax    0x2b                43
(gdb) continue
Continuing.
[Capa C] Valor convertido y sumado por ASM: 43
Resultado final en main c: 43
[Inferior 1 (process 27254) exited normally]
(gdb)
```

Bibliografía

- Guía rápida para uso del debugger GDB - Departamento de Computación FCEFyN.
- Enunciado TP#2: Calculadora de índices GINI - Javier Jorge.